Министерство цифрового развития, связи и массовых коммуникаций
Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования «Сибирский государственный университет
телекоммуникаций и информатики»
(СибГУТИ)

Кафедра прикладной математики и кибернетики

Операционные системы

Лабораторная работа №8

Выполнили: Студенты 3-го курса,
группы ИП-111
Гердележов Даниил Дмитриевич
Махаева Алина Витальевна
Клепче Георгий Васильевич

Проверил преподаватель:
Малков Евгений Александрович

Новосибирск, 2023

Оглавление

Задание: 3
Цель: 3
Ход работы: 4
Вывод: 5
Листинг: 6
Pthreads.cpp 6
thread.cpp 7
Makefile 8

Задание: программно реализуйте вычисление суммы последовательности чисел на основе последовательного кода, интерфейсов Pthreads и C++11 . Сравните время вычислений.

Цель: знакомство с программными интерфейсами управления потоками в Linux

Ход работы:

Разработаем две программы для вычисления суммы последовательности чисел пусть это будет сумма чисел от 0 до 1 000 000 на основе последовательного кода, интерфейсов Pthreads и C++11 (код программ представлен в листинге).

Напишем makefile для компиляции программ (код представлен в листинге).

```
nothomepc@DESKTOP-M12GO9R:~/code/lab8$ make
g++ -o Pthreads Pthreads.cpp
g++ -o thread thread.cpp
nothomepc@DESKTOP-M12GO9R:~/code/lab8$ ./thread
Сумма: 1784293664
Время выполнения: 0.00515239 секунд
nothomepc@DESKTOP-M12GO9R:~/code/lab8$ ./Pthreads
Сумма: 1784293664
Время выполнения: 0.0070068 секунд
nothomepc@DESKTOP-M12GO9R:~/code/lab8$
```

Рис. 1 – Компиляция программ и результат работы.

Что бы уменьшить влияние погрешности измерений на конечный вывод о скорости работы программ возьмем среднее значение за 10 измерений:

	./thread	./Pthreads
	0,00595877	0,01540210
	0,00637165	0,00400183
	0,00499660	0,00326712
	0,00282900	0,01328630
	0,00858768	0,00589999
	0,00571146	0,01916830
	0,00554543	0,01183010
	0,00458841	0,00372565
	0,00599966	0,01234820
	0,01087030	0,00647428
Ср. знач	0,00614590	0,00954039

Рис. 2 – Результаты 10 измерений и среднее значение выполнения вычислений.

Вывод:

- Мы программно реализовали вычисление суммы последовательности чисел от 0 до 1 000 000 на основе последовательного кода, интерфейсов Pthreads и C++11.
- Сделали по 10 измерений времени вычисления и взяли среднее значение (рис. 2), на основе полученных нами данных можно сделать вывод о том, что вычисления в программе Pthreads выполнялись на 55.23% дольше.

Листинг:

Pthreads.cpp

```
#include <iostream>
#include <vector>
#include <pthread.h>
#include <numeric>
#include <chrono>

// Структура для передачи аргументов в поток
struct ThreadArgs {
    const std::vector<int>& numbers;
    size_t start;
    size_t end;
    int result;
};

// Функция, которую будут выполнять потоки
void* sumRange(void* args) {
    ThreadArgs* threadArgs = static_cast<ThreadArgs*>(args);
    threadArgs->result = std::accumulate(threadArgs->numbers.begin() +
threadArgs->start, threadArgs->numbers.begin() + threadArgs->end, 0);
    return nullptr;
}

int main() {
    std::vector<int> numbers(1000001);
    std::iota(numbers.begin(), numbers.end(), 0); // Заполняем вектор числами от
0 до 1000
    int numThreads = 4; // Количество потоков

    std::vector<pthread_t> threads(numThreads);
    std::vector<ThreadArgs*> threadArgs(numThreads);

    size_t chunkSize = numbers.size() / numThreads;

    auto start = std::chrono::high_resolution_clock::now();

    // Создаем и запускаем потоки
    for (int i = 0; i < numThreads; i++) {
        threadArgs[i] = new ThreadArgs{numbers, i * chunkSize, (i == numThreads -
1) ? numbers.size() : (i + 1) * chunkSize, 0};
        pthread_create(&threads[i], nullptr, sumRange, threadArgs[i]);
    }

    // Ждем завершения всех потоков
    for (int i = 0; i < numThreads; i++) {
        pthread_join(threads[i], nullptr);
    }

    // Суммируем частичные суммы
```

```
    int totalSum = 0;
    for (int i = 0; i < numThreads; i++) {
        totalSum += threadArgs[i]->result;
        delete threadArgs[i]; // Освобождаем выделенную память
    }

    auto end = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> duration = end - start;

    std::cout << "Сумма: " << totalSum << std::endl;
    std::cout << "Время выполнения: " << duration.count() << " секунд" <<
std::endl;

    return 0;
}
```

thread.cpp

```
#include <iostream>
#include <thread>
#include <vector>
#include <numeric>
#include <chrono>

// Функция, которую будут выполнять потоки
void sumRange(const std::vector<int>& numbers, size_t start, size_t end, int&
result) {
    result = std::accumulate(numbers.begin() + start, numbers.begin() + end, 0);
}

int main() {
    std::vector<int> numbers(1000001);
    std::iota(numbers.begin(), numbers.end(), 0); // Заполняем вектор числами от
0 до 1000
    int numThreads = 4; // Количество потоков

    std::vector<std::thread> threads(numThreads);
    std::vector<int> partialSums(numThreads);

    size_t chunkSize = numbers.size() / numThreads;

    auto start = std::chrono::high_resolution_clock::now();

    // Создаем и запускаем потоки
    for (int i = 0; i < numThreads; i++) {
        size_t start = i * chunkSize;
        size_t end = (i == numThreads - 1) ? numbers.size() : (start +
chunkSize);
        threads[i] = std::thread(sumRange, std::ref(numbers), start, end,
std::ref(partialSums[i]));
```

```
    }

    // Ждем завершения всех потоков
    for (int i = 0; i < numThreads; i++) {
        threads[i].join();
    }

    // Суммируем частичные суммы
    int totalSum = std::accumulate(partialSums.begin(), partialSums.end(), 0);

    auto end = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> duration = end - start;

    std::cout << "Сумма: " << totalSum << std::endl;
    std::cout << "Время выполнения: " << duration.count() << " секунд" <<
std::endl;

    return 0;
}
```

Makefile

```
CC = g++

all: Pthreads thread

Pthreads:
    $(CC) -o Pthreads Pthreads.cpp

thread:
    $(CC) -o thread thread.cpp

clean:
    rm -f Pthreads thread
```